FoodZeep — Full Product Engineering Deck (Production-Grade Food Delivery Platform)

Version

v2.0 — Product + Engineering + Deployment + Scale Blueprint

Purpose of This Document

This is not a resume project document.

This is a real product engineering blueprint for building FoodZeep like a production-grade platform similar in long-term vision to Swiggy / Zomato / food commerce systems.

This document is written for:

- Founders
- Backend Developers
- Frontend Developers
- DevOps Engineers
- QA Engineers
- Product Managers
- Interview Panels
- Mentors
- New Developers Joining the Team
- Future Investors / Technical Reviewers

This document should answer:

- What are we building?
- Why are we building it?
- How should it be built?
- Which tools should be used?
- What should happen first?
- What scales later?
- What belongs where?
- How do we go from localhost → production → real users?

This is the source of truth.

---

1. Product Vision

Product Name

FoodZeep

Product Category

Food Commerce + Food Redistribution + Inventory + Delivery Platform

Long-Term Vision

Build a full-stack food platform where:

- Restaurants / sellers manage food inventory
- Buyers discover and order food
- NGOs can request surplus food donations
- Delivery partners complete deliveries
- Admin manages the full ecosystem

Final vision:

A hybrid of:

- Swiggy
- Zomato
- Blinkit operational thinking
- Food donation ecosystem

This is not a CRUD project.

This is a scalable business platform.

---

2. Problem Statement

Industry Problems

Restaurants face:

- surplus food waste
- poor expiry tracking
- poor inventory visibility
- no seller-level food control
- manual coordination issues

Customers face:

- lack of transparency
- pricing inconsistency
- poor delivery tracking

NGOs face:

- difficult access to excess food donations

Admin teams face:

- no centralized control
- poor seller accountability

---

3. Product Solution

FoodZeep solves this by providing:

Seller Platform

- authentication
- food listing
- food updates
- expiry tracking
- stock visibility
- delete unavailable items
- dashboard analytics

Buyer Platform

- browse nearby food
- filter by veg / non-veg / pricing
- order placement
- live order tracking
- payment system

NGO Platform

- donation requests
- surplus food pickup

Delivery Platform

- pickup assignment
- delivery tracking
- status updates

Admin Platform

- seller management
- fraud monitoring
- reporting
- override permissions
- analytics

---

4. Product Modules

Core Modules

Authentication

User Management

Food Management

Restaurant Management

Buyer Ordering

Cart System

Payment System

Delivery Tracking

Notifications

Reviews & Ratings

Search & Filtering

Dashboard & Analytics

Admin Panel

Support System

Refund System

Logs & Monitoring

Deployment & DevOps

Everything matters equally.

---

5. User Roles

Admin

System-wide control.

Can:

- manage all users
- manage restaurants
- override food ownership
- access analytics
- fraud detection
- support resolution

Seller / Restaurant Owner

Can:

- register restaurant
- add food
- update food
- delete food
- manage stock
- track orders
- manage revenue

Buyer / Customer

Can:

- browse food
- place orders
- pay online
- track orders
- rate food

Delivery Partner

Can:

- accept delivery jobs
- update delivery status
- track earnings

NGO Partner

Can:

- request surplus food
- donation tracking

---

6. Development Phases

Phase 1 — Foundation

- requirements
- database design
- backend architecture
- authentication
- food CRUD

Phase 2 — Product Growth

- buyer flows
- restaurant flows
- payments
- delivery
- notifications

Phase 3 — Production Engineering

- caching
- queues
- scaling
- monitoring
- deployment
- CDN
- cloud infra

Phase 4 — Enterprise Scale

- microservices
- event-driven architecture
- advanced analytics
- AI recommendations

---

7. Complete Tech Stack

Frontend

Web

- React
- Next.js (preferred for scale)
- Tailwind CSS
- TypeScript
- Redux Toolkit / Zustand
- React Query
- React Router

Mobile

- React Native (future)

Backend

Core

- Node.js
- Express.js / Fastify
- TypeScript (production preferred)

Security

- JWT
- Refresh Tokens
- bcrypt
- Helmet
- Rate Limiting
- CORS

Validation

- Joi / Zod / Yup

File Upload

- Multer
- Cloudinary / AWS S3

Background Jobs

- BullMQ
- Redis

Real-time

- Socket.IO

Database

Primary

- MySQL / PostgreSQL

Cache

- Redis

Search (future)

- Elasticsearch

DevOps

- Docker
- Docker Compose
- Nginx
- PM2
- GitHub Actions
- CI/CD pipelines

Cloud

- AWS / GCP / Azure

Services

- EC2
- RDS
- S3
- CloudFront
- Route53
- Load Balancer
- CloudWatch

Monitoring

- Winston
- Morgan
- Sentry

- Grafana
- Prometheus

---

8. Domain + Hosting

Example Domain

foodzeep.com

Hosting Layers

Frontend Hosting

- Vercel
- Netlify
- AWS S3 + CloudFront

Backend Hosting

- AWS EC2
- Railway
- Render
- VPS

Database Hosting

- AWS RDS
- PlanetScale
- Neon

DNS

- GoDaddy
- Namecheap
- Route53

Production means real domain + SSL + monitoring.

---

9. Full Development Flow (Professional Order)

Never start by writing controllers.

Correct order:

Step 1

Requirements

Step 2

Product Flow Mapping

Step 3

Database Design

Step 4

API Contract Design

Step 5

Folder Architecture

Step 6

Security Planning

Step 7

Validation Strategy

Step 8

Controller/Service/Model Development

Step 9

Testing

Step 10

Deployment

Step 11

Monitoring

Step 12

Scaling

This is real engineering.

---

10. Backend Architecture

Folder Structure

```
src/
│
├── modules/
│   ├── auth/
│   ├── users/
│   ├── food/
│   ├── orders/
│   ├── payments/
│   ├── restaurants/
│   ├── delivery/
│   └── notifications/
│
├── middleware/
├── validators/
├── utils/
├── config/
├── jobs/
├── logs/
├── docs/
├── tests/
├── app.js
└── server.js
```

---

11. Layer Responsibilities

Routes

Only route mapping

Controller

Request/response only

Service

Business rules

Model

Database only

Middleware

Reusable cross-cutting logic

Validator

Input safety

Utils

Reusable helpers

Jobs

Background processing

This separation is mandatory.

---

12. Database Design (Current + Future)

Current Tables

- users
- food_items

Future Tables

- restaurants
- carts
- orders
- order_items
- payments
- delivery_partners
- delivery_tracking
- notifications
- reviews
- ratings
- coupons
- refunds
- support_tickets
- audit_logs

Database is product strategy.

---

13. API Documentation Strategy

Use:

- Swagger
- OpenAPI specs
- Postman collections

Every API must be documented.

No undocumented APIs.

---

14. Validation Strategy

Every request must be validated before service.

Examples:

- email valid
- quantity integer
- price positive
- expiry valid
- role allowed

Validation prevents production disasters.

---

15. Error Handling Strategy

Use:

Global Error Middleware

Not endless try/catch chaos.

Error categories:

- validation
- DB
- auth
- payment
- external API
- infra

Standardized responses only.

---

16. Logging Strategy

Track:

- login attempts
- order creation
- payment failures
- food deletion
- suspicious behavior
- admin overrides

Logs are production memory.

---

17. Testing Strategy

Required

- unit tests
- integration tests
- API tests
- regression testing

Tools:

- Jest
- Supertest
- Postman

Never deploy untested critical flows.

---

18. CI/CD Strategy

Every push should trigger:

- lint check
- tests
- build validation
- deployment pipeline

Use:

GitHub Actions

No manual chaos.

---

19. Deployment Strategy

Localhost

Development

Staging

Internal testing

Production

Real users

Never deploy directly from localhost mindset.

---

20. Scaling Concepts

Must Know

- load balancer
- API gateway
- CDN
- caching
- latency
- horizontal scaling
- vertical scaling
- DB indexing
- connection pooling
- Redis
- message queues
- queues vs sync
- background workers
- rate limiting

This is backend maturity.

---

21. Security Strategy

Required

- JWT access tokens
- refresh tokens
- password hashing

- RBAC
- SQL injection prevention
- XSS prevention
- CSRF awareness
- secure headers
- audit logs

Security is not optional.

---

22. Product Analytics

Track:

- active users
- daily orders
- seller performance
- top restaurants
- refund percentages
- failed payments

Data drives product decisions.

---

23. Interview Explanation

One-line:

FoodZeep is a production-grade scalable food commerce and redistribution platform designed with modular backend architecture, secure authentication, role-based authorization, delivery-ready infrastructure, and cloud deployment planning similar to Swiggy/Zomato systems.

---

24. Final Engineering Rule

Never ask:

“How do I write this API?”

Ask:

“How should this product survive 10 lakh users?”

That is real backend engineering.

End of Master Product Engineering Deck